CloudVault

Professional MERN Cloud Storage Platform

CloudVault is a responsive, high-fidelity cloud storage and file collaboration application built on the MERN stack (MongoDB, Express, React, Node.js). Drawing design inspiration from premium modern dashboards, the application implements presigned S3 direct uploads, advanced file versioning, bulk operations, collaborative sharing for registered and unregistered users, and an anonymous shared web-preview interface.

---

1. System Architecture & Tech Stack

CloudVault utilizes a decoupled, client-server architecture. To ensure optimal performance and avoid bottlenecking compute servers, file uploads and downloads bypass backend server memory entirely by streaming directly between the client browser and the S3 storage bucket.

Technology Stack

- **Frontend**: React (v19), Tailwind CSS, Lucide icons, Vite.
- **Backend**: Node.js, Express, JSON Web Tokens (JWT) for stateless sessions, Mongoose ODM.
- **Database**: MongoDB Atlas.
- **Object Storage**: AWS S3 Bucket (configured with CORS policies allowing direct client PUT/GET requests), with a fully functional local filesystem upload fallback (`uploads/` directory) for development environments.
- **Process Manager**: PM2 process manager running backend services.
- **Server/Web Proxy**: Nginx reverse proxy directing API calls to the Node.js port (`5001`) and serving static React assets. Secured via Let's Encrypt SSL/TLS certificates.

---

2. Core Features & Capabilities

Ø=Ý Authentication & Identity Management

- **Credentials Flow**: Secure registration and login views with password hashing (bcrypt) and session persistence via JWT.
- **OTP Password Reset**: Forgot password flow generating email-dispatched OTP verification codes to reset passwords securely.
- **Replica Google OAuth Popup**:
- Clicking "Continue with Google" launches a centered popup mimicking `accounts.google.com`.
- Integrates client-side profile caching (`localStorage`) to remember previously signed-in accounts in a "Saved accounts" widget.
- Queries database users with input debouncing.
- Passes chosen credentials back to the main app via secure `window.opener.postMessage` events and closes itself.

Ø=ÜÁ File Organization & Directory Navigation

- **Drag-and-Drop Uploader**: A large, touch-sensitive file drop zone showing live transmission speed and progress sliders.
- **Interactive Folders**: Group documents inside folders. Navigating folder scopes dynamically updates views, with breadcrumb coordinates managing return paths.
- **Starred Collection**: Mark documents as important to compile them into a quick-access "Starred" dashboard tab.
- **Two-Stage Trash Bin**: Soft-delete items to the "Trash Bin" to remove them from directories. Trashed files restrict standard operations, showing only "Restore" or "Permanently Delete" buttons.
- **Header Search**: Live header search bar scanning document names globally across all folder scopes.

Ø=Üe Collaborative Sharing & Auto-linking

- **Specific Collaborative Shares**: Share folders or documents with other emails, assigning roles (Viewer / Editor).
- **Unregistered User Support**: Files and folders can be shared with emails that do not yet have a CloudVault account (`user: null` in MongoDB).
- **Automatic Registration Linking**: When a new user registers, the database registration hook scans for pending shares matching the user's email and automatically updates the records to link their new user `_id` instantly.
- **Access Revocation**: Owners can instantly revoke file/folder access in the share list.

Ø=Ý Advanced File Versioning

- **Automatic Version Tracking**: Uploading a file with an identical name inside the same folder automatically archives the older file to the file's `versions` history array and uploads the new file to the active S3 key.
- **Timeline Drawer**: Opens a side drawer listing past revisions with version numbers, S3 keys, file sizes, and upload timestamps.
- **Rollback Recovery**: Click "Restore" on any historical version to restore it as the active document.
- **Purge History**: Delete specific historical versions permanently from S3 to reclaim storage space.

Ø=Üæ Bulk Operations

- **Multi-Selection Grid**: Multi-select files using checkboxes in the file grid to run batch actions.
- **Bulk Move**: Move selected files into a chosen folder in a single action.
- **Bulk Download**: Compresses selected files into a single `.zip` file on the fly and downloads the archive directly to the device.
- **Bulk Soft-Trash / Restore / Hard-Delete**: Recycle, restore, or permanently delete selected files in batches.

Ø=Ý Public Web Shared-Preview Page

- **Stateless Access Token (`emailToken`)**: Sharing a document generates a 7-day cryptographic JWT (`emailToken`) containing the recipient's email and `fileId`.
- **Public Preview Route**: Serves a public frontend page (`/shared-preview/:fileId?emailToken=...`) allowing unregistered recipients to access documents without logging in or signing up.
- **Dynamic Inline Viewers**: Renders images, text/code contents, video/audio players, and PDFs (utilizing a mobile fallback layout to avoid viewport iframe scrolling bugs on iOS/Android).
- **Download Button**: A direct download button allowing guests to download the file directly to their local

machine.

- **Revocation Security**: If the owner revokes the share, subsequent requests to the preview page instantly return a `403 Access Denied` error.

---

3. Directory Structure

```
micro-google-drive/
% % %  backend/
%    % % %  config/              # MongoDB connection setups
%    % % %  controllers/         # API controllers (fileController.js, authController.js)
%    % % %  middleware/          # JWT protect token auth checks
%    % % %  models/              # Mongoose Schemas (User.js, File.js, Folder.js)
%    % % %  routes/              # Express API Routes (authRoutes.js, fileRoutes.js)
%    % % %  services/            # Core external integrations (emailService.js)
%    % % %  uploads/             # Local fallback directory for mock S3 uploads
%    % % %  server.js            # App listener bootstrapper
%    % % %  test-*.js            # Automated E2E verification test suites
% % %  frontend/
%    % % %  dist/                # Output compiled static production assets
%    % % %  src/
%    %    % % %  components/      # Sidebar, ShareModal, DropzoneUpload, FileGrid
%    %    % % %  context/         # AuthContext.jsx (Axios API setup, Themes)
%    %    % % %  pages/           # Pages (Dashboard.jsx, AuthPage.jsx, SharedPreviewPage.jsx)
%    %    % % %  App.jsx          # Frontend client router controller
%    %    % % %  main.jsx         # DOM anchor bootstrapper
%    % % %  vite.config.js       # Vite build configurations
%    % % %  package.json         # Client dependency configuration
% % %  deploy-package.sh        # Local build and compression script
% % %  ec2-setup.sh             # Remote shell configuration script for Ubuntu EC2
```

---

4. Production Deployment & Live Server Configuration

All modifications are deployed live to AWS at cloudvault-anurag.duckdns.org:

- **Nginx Reverse Proxy**:
- Directs `/` traffic to the static React bundle at `/var/www/cloudvault/frontend-dist`.
- Proxies `/api` traffic to Node.js backend port `5001`.
- Configured with `try_files $uri $uri/ /index.html;` to allow client-side SPA routing for `/shared-preview/:fileId` URLs.
- **SSL Certificate**: Let's Encrypt SSL certificates (configured via Certbot) providing secure HTTPS connections.
- **PM2 Process Manager**: Processes are managed as background daemons, automatically reloading on crashes or server restarts.
- **Security & Storage Quota**: Users are restricted to a strict **20 GB limit**, preventing S3 bucket exhaustion.
